

Algorithms

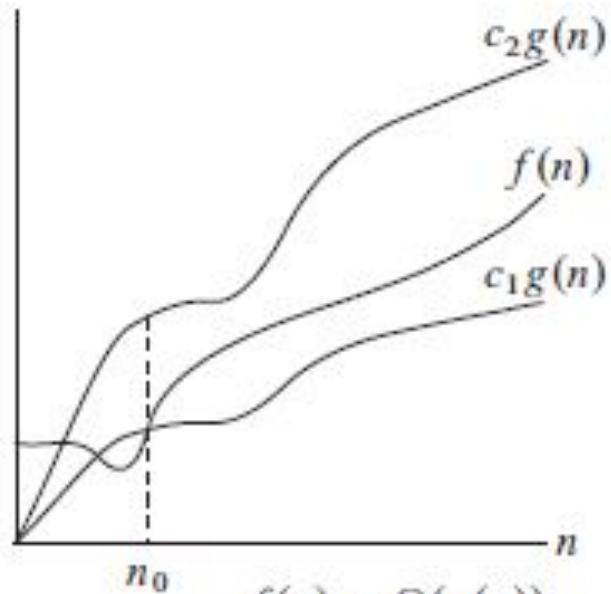
Lecture 1.1

Dr. Md Moniruzzaman

Reference Books

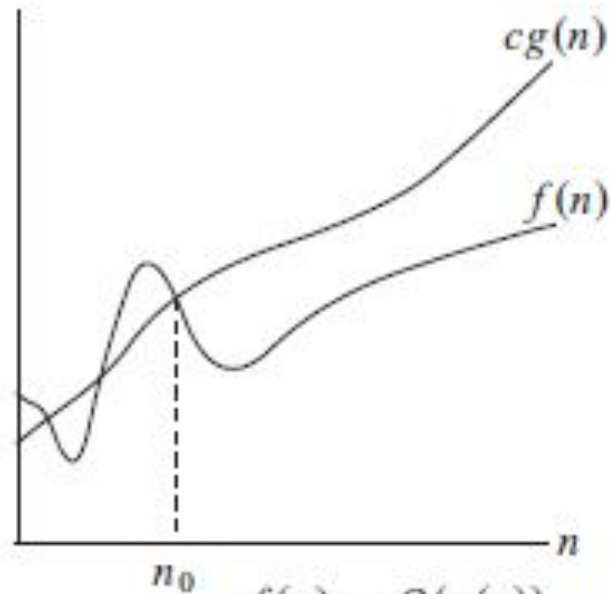
- 1. Introduction to Algorithms (Cormen, Leiserson, Rivest, and Stein)
- 2. Algorithms (Dasgupta, Papadimitriou, and Vazirani)
- 3. Algorithms (Erickson)

Complexity Analysis



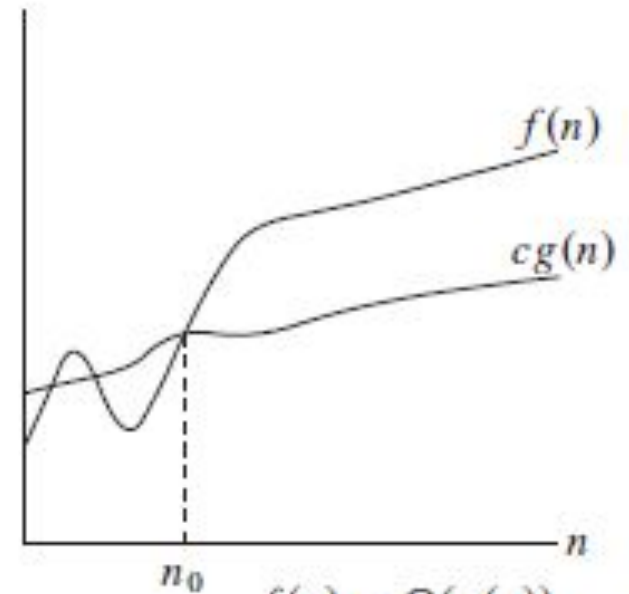
$$f(n) = \Theta(g(n))$$

(a)



$$f(n) = O(g(n))$$

(b)



$$f(n) = \Omega(g(n))$$

(c)

Complexity Analysis

$$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that} \\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\} .^1$$

$$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that} \\ 0 \leq f(n) \leq c g(n) \text{ for all } n \geq n_0\} .$$

$$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that} \\ 0 \leq c g(n) \leq f(n) \text{ for all } n \geq n_0\} .$$

Sorting

Bubble Sort

Bubble Sort:

```
for(i=0; i<m-1; i++){  
    for(j=0; j<m-i-1; j++){  
        if(count[j] > count[j+1]){  
            swap(ar, j, j+1);  
        }  
    }  
}
```

Selection Sort

```
public class Selection
{
    public static void sort(Comparable[] a)
    { // Sort a[] into increasing order.
        int N = a.length;           // array length
        for (int i = 0; i < N; i++)
        { // Exchange a[i] with smallest entry in a[i+1...N).
            int min = i;             // index of minimal entr.
            for (int j = i+1; j < N; j++)
                if (less(a[j], a[min])) min = j;
            exch(a, i, min);
        }
    }
    // See page 245 for less(), exch(), isSorted(), and main().
}
```

Insertion Sort:

```
public class Insertion
{
    public static void sort(Comparable[] a)
    { // Sort a[] into increasing order.
        int N = a.length;
        for (int i = 1; i < N; i++)
        { // Insert a[i] among a[i-1], a[i-2], a[i-3]... ..
            for (int j = i; j > 0 && less(a[j], a[j-1]); j--)
                exch(a, j, j-1);
        }
    }
    // See page 245 for less(), exch(), isSorted(), and main().
}
```


Merge Sort: (Sort Procedure)

```
private static void sort(Comparable[] a, int lo, int hi)
{ // Sort a[lo..hi].
  if (hi <= lo) return;
  int mid = lo + (hi - lo)/2;
  sort(a, lo, mid);      // Sort left half.
  sort(a, mid+1, hi);    // Sort right half.
  merge(a, lo, mid, hi); // Merge results (code on page 271).
}
```

Merge Sort: (Merge Procedure)

```
public static void merge(Comparable[] a, int lo, int mid, int hi)
{ // Merge a[lo..mid] with a[mid+1..hi].
  int i = lo, j = mid+1;

  for (int k = lo; k <= hi; k++) // Copy a[lo..hi] to aux[lo..hi].
    aux[k] = a[k];

  for (int k = lo; k <= hi; k++) // Merge back to a[lo..hi].
    if      (i > mid)           a[k] = aux[j++];
    else if (j > hi )           a[k] = aux[i++];
    else if (less(aux[j], aux[i])) a[k] = aux[j++];
    else                       a[k] = aux[i++];
}
```

Useful links

- <https://www.bigocheatsheet.com/>
- <https://visualgo.net/en>
- <https://www.hackerearth.com/practice/algorithms/sorting/bubble-sort/visualize/>

Problems to solve using sorting:

- UVA 612
- Codeforces 984A